FOODRESCUEAGENT

Guida al prompt del chatbot IA e ai test

Documento per il gruppo di progetto | 10 novembre 2026

1. Che cosa abbiamo preparato

Abbiamo progettato un prompt di sistema per un assistente IA dedicato all'organizzazione di eventi alimentari e 10 casi di test per controllarne il comportamento. Il chatbot deve stimare quantità ragionevoli di ingredienti, ridurre gli sprechi e, quando richiesto, proporre anche un menù.

Il prompt è dinamico: le istruzioni generali restano uguali, mentre il modello riceve ogni volta i dati dell'evento e il messaggio dell'utente. Non occorre scrivere un nuovo prompt per ciascuna cena o festa.

Le tre modalità previste

Modalità	Quando si attiva	Risultato atteso
Calcolo ingredienti	L'utente ha già scelto il menù	Quantità per ingrediente e porzioni
Proposta menù	L'utente chiede idee o ricette	Menù proposto e relative quantità
Modifica menù	L'utente cambia un piatto	Menù aggiornato e ingredienti ricalcolati

Esempio: "Suggeriscimi un buffet per 40 persone". Il modello propone alcune pietanze, distribuisce le porzioni tra le preparazioni e restituisce un elenco aggregato degli ingredienti in JSON.

2. Dove salvare i file nel repository

Repository: github.com/AlessiaDeMichele/FoodRescueAgent

Proponiamo di aggiungere una nuova cartella AI nella radice del progetto:

```
FoodRescueAgent/
  AI/
    system_prompt.txt
    test_cases.json
    README.md            (facoltativo)
```

- system_prompt.txt: contiene il testo integrale del prompt di sistema, con obiettivi, vincoli e formato JSON.
- test_cases.json: contiene le richieste campione e i controlli attesi.
- README.md: può spiegare l'uso dei due file agli sviluppatori.

Su GitHub: Add file → Create new file, digitare ad esempio AI/system_prompt.txt, incollare il testo e selezionare Commit changes. Ripetere per i test.

3. Come funziona il chatbot

Flusso previsto: l'utente scrive una richiesta → il backend recupera le informazioni dell'evento → invia al modello il prompt di sistema e i dati dinamici → riceve una risposta JSON → la valida → la mostra al sito e, se appropriato, la salva nel database.

Quali dati vanno inviati al modello

- Numero di invitati e, se noto, suddivisione adulti/bambini.
- Tipo di evento (cena, pranzo, buffet, aperitivo...), menù esistente o richiesta di proposta.
- Budget, preferenze, allergie, restrizioni alimentari, ingredienti disponibili e quantità.
- Per le modifiche successive: menù aggiornato o storico rilevante della chat.

Esempio di input dinamico

```
{
  "numero_invitati": 40,
  "tipo_evento": "buffet serale",
  "menu": ["pizzette", "panini", "torta"],
  "budget": 150,
  "ingredienti_disponibili": [],
  "restrizioni_alimentari": []
}
```

Questo è un esempio di dati della richiesta, non il prompt di sistema: i due contenuti vengono inviati al modello con ruoli distinti.

4. Che cosa deve contenere la risposta

Il modello deve restituire soltanto JSON valido, senza frasi introduttive né blocchi Markdown. Il formato consente al backend di analizzare i risultati in modo automatico.

```
{
  "stato": "ok",
  "modalita": "proposta_menu",
  "numero_invitati": 6,
  "tipo_evento": "cena",
  "budget": null,
  "menu": [
    {"piatto": "Pasta al pomodoro",
     "descrizione": "Primo semplice", "porzioni_previste": 6}
  ],
  "ingredienti": [
    {"nome": "Pasta secca", "quantita_totale": 420,
     "quantita_disponibile": null,
     "quantita_mancante": null, "unita": "g"}
  ],
  "ipotesi": ["6 adulti, cena con più portate"],
  "avvertenze": []
}
```

I valori riportati sono illustrativi. Gli ingredienti in produzione devono essere completi e coerenti con tutte le pietanze del menù.

Regole principali: quantità numeriche non negative, unità coerenti (g, ml, pz), ingredienti uguali aggregati e indicazione delle ipotesi. Se la disponibilità di un ingrediente è sconosciuta, le quantità disponibile e mancante devono essere null (non zero). Se mancano dati essenziali, lo stato è dati_insufficienti.

5. Come testare il prompt

Il file test_cases.json contiene 10 scenari. Ogni test ha un campo input, una modalità attesa e una serie di verifiche. I controlli sono criteri di valutazione, non risposte già generate.

Test	Scenario	Cosa verificare
1	Cena per 6	Porzioni coerenti con più portate
2	Buffet per 40	Niente porzioni complete di ogni piatto
3	Proposta economica per 10	Menù suggerito e ingredienti
4	Pranzo vegetariano per 12	Rispetto delle preferenze
5	Buffet con budget di 100 €	Nessun prezzo inventato
6	Pasta con scorte disponibili	Sottrazione corretta delle scorte note
7	Festa con adulti e bambini	Ipotesi sulle porzioni differenziate
8	Buffet con allergie	Avvertenze e restrizioni rispettate
9	Sostituzione di un piatto	Menù precedente mantenuto e ricalcolo
10	Richiesta troppo generica	Stato dati_insufficienti

Procedura pratica

- 1. Configurare un modello IA con il testo di system_prompt.txt come messaggio di sistema.
- 2. Per ciascun caso, inviare il campo input come messaggio dell’utente. Nel caso 9 fornire anche contesto_precedente.
- 3. Verificare che la risposta sia JSON valido e che i campi rispettino la struttura concordata.
- 4. Controllare manualmente plausibilità delle porzioni, unità, somme, restrizioni e ingredienti.
- 5. Annotare gli errori; modificare il prompt; ripetere gli stessi test per confrontare il risultato.

Le quantità non vanno valutate soltanto perché il JSON è formalmente corretto: una risposta può essere valida sintatticamente ma contenere stime irrealistiche.

Scheda minima per annotare i risultati

Campo	Esempio di compilazione
Test	2 - Buffet per 40
JSON valido?	Sì / No
Modalità corretta?	Sì / No
Quantità ragionevoli?	Sì / No + nota
Problemi	Es.: troppa pizza rispetto ai rustici
Esito	Superato / Da correggere

6. Relazione con il database esistente

Nel repository è presente uno schema SQL con le tabelle evento, ingrediente, ingrediente_evento, chat_ia e messaggio_ia. Queste sono rilevanti per il chatbot:

Tabella	Ruolo previsto
evento	Numero invitati, budget e organizzatore
ingrediente	Catalogo ingredienti con id_ingrediente e nome
ingrediente_evento	Ingredienti e quantità associati a un evento
chat_ia	Una chat per evento tramite id_evento univoco
messaggio_ia	Testo, autore e data dei messaggi della chat

Attenzione: nello schema SQL consultato non sono presenti campi specifici per menù, tipologia dell'evento, allergie o scorte disponibili. Questi dati dovranno essere trasmessi dall'applicazione o richiederanno una scelta progettuale sul database. Inoltre l'output JSON non coincide direttamente con lo schema SQL: il backend deve convertire i nomi degli ingredienti negli ID e controllare le unità prima di salvare.

7. Integrazione nel sito: cosa manca

Il frontend attuale è un sito statico HTML/CSS/JavaScript con integrazione Supabase e modalità demo. Il README descrive login, profili, mappa e gestione centri. Il prompt e i test non equivalgono a un chatbot già integrato e funzionante.

- Creare un'interfaccia chat collegata agli eventi e una chiamata backend sicura al modello.
- Tenere la chiave API del modello solo sul server (ad esempio in una Supabase Edge Function); mai nel JavaScript pubblico.
- Caricare il prompt nel backend e combinarlo con i dati di evento e conversazione.
- Usare JSON Schema / structured output quando disponibile e validare nuovamente nel backend.
- Persistire messaggi e risultati solo dopo controlli di formato, permessi e coerenza.
- Gestire errori, risposte incomplete, limiti del modello e richieste di chiarimento.

8. Limiti e decisioni da concordare

- Budget: il modello non deve fingere di conoscere prezzi aggiornati; per verificare il costo reale servirà un listino attendibile.
- Allergie: nessuna risposta IA può garantire sicurezza alimentare; verificare etichette e contaminazioni.
- Porzioni: gli intervalli indicati nel prompt sono stime iniziali; il team deve validarli con casi realistici.
- Scorte: quando non sono note, il JSON riporta null, evitando di confondere "dato mancante" e "quantità zero".
- Formato: concordare la struttura JSON definitiva con chi sviluppa il backend prima dell'integrazione.

9. Checklist di consegna

- Creare AI/system_prompt.txt con il prompt completo e aggiornato.
- Creare AI/test_cases.json con i 10 scenari.
- Verificare che test_cases.json sia JSON valido.
- Condividere con il gruppo il contratto JSON e i punti ancora da implementare.
- Dopo il collegamento al modello, eseguire e documentare i test: nessun test è ancora dichiarato superato.

Stato del lavoro: progettazione del comportamento dell'IA, struttura dell'output e piano di test definiti; integrazione, esecuzione dei test e verifiche end-to-end da completare.